

ДИСЦИПЛИНА	Программирование (полное наименование дисциплины без сокращений)
ИНСТИТУТ	Институт тонких химических технологий им. М.В. Ломоносова
КАФЕДРА	Кафедра информационных систем в химической технологии полное наименование кафедры)
ВИД УЧЕБНОГО МАТЕРИАЛА	Практические занятия (в соответствии с пп.1-11)
ПРЕПОДАВАТЕЛЬ	Серебренников Н.П., Чиреев В.В. (фамилия, имя, отчество)
СЕМЕСТР	2-й семестр, 1-й курс бакалавриата, 2024-2025 учебный год (указать семестр обучения, учебный год)

Практическая работа № 4. Условные переходы, циклы и обработка исключений в Python

Цель работы

Изучить основы работы с условными переходами, множественными ветвлениями, циклами и обработкой исключений в Python. Закрепить навыки работы с последовательностями и их операциями.

Задания

1. Условные переходы и множественные ветвления.

- Создайте список из 5 чисел.
- Напишите программу, которая проверяет каждый элемент списка:
 - Если число больше 10, выведите сообщение: "Число [число] больше 10".
 - Если число равно 10, выведите сообщение: "Число [число] равно 10".
 - Если число меньше 10, выведите сообщение: "Число [число] меньше 10".
- Используйте конструкцию `if-elif-else`.

```
numbers = [5, 10, 15, 3, 8]
for num in numbers:
    if num > 10:
        print(_____)
    elif _____:
```

```
        print(_____)
else:
    print(_____)
```

- Напишите программу для выполнения арифметических операций с проверкой на деление на ноль.
- Используйте `if -else`.

```
a = int(input("Введите первое целое число: "))
b = int(input("Введите второе целое число: "))
if _____:
    print("Деление на ноль!")
else:
    sum = _____
    difference = _____
    product = _____
    quotient = round(_____)

    print("Сумма:", sum)
    print("Разность:", difference)
    print("Произведение:", product)
    print("Частное:", quotient)
```

2. Циклы (for и while)

- Создайте список из 10 чисел.
- Используя цикл `for`, найдите сумму всех элементов списка.
- Используя цикл `while`, найдите произведение всех элементов списка.
- Выведите результаты на экран.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```

# Сумма через for
sum_result = 0
for num in numbers:
    _____

# Произведение через while
product_result = 1
i = 0
while i < len(numbers):
    _____

print(f"Сумма: {sum_result}")
print(f"Произведение: {product_result}")

```

3. Управление циклами (break, continue)

- Создайте список из 10 чисел.
- Используя цикл `for`, найдите первое число, которое делится на 3 без остатка.
- Если такое число найдено, выведите его и завершите цикл с помощью `break`.
- Если число не делится на 3, пропустите его с помощью `continue`.

```

numbers = [1, 2, 4, 5, 6, 7, 8, 9, 10, 12]
for num in _____:
    if _____:
        print("Первое число, делящееся на 3:", num)
        break
    else:
        continue

```

4. Множественные исключения

- Создайте словарь, где ключи — это имена студентов, а значения — их оценки (например: {"Alice": 85, "Bob": 90}).

- Напишите программу, которая запрашивает у пользователя имя студента и выводит его оценку.
- Обработайте следующие исключения:
 - Если студента нет в словаре, выведите сообщение: "Студент [имя] не найден".
 - Если пользователь вводит не строку, а число, выведите сообщение: "Имя студента должно быть строкой".

```
grades = {"Alice": 85, "Bob": 90}

try:
    name = input("Введите имя студента: ")
    if not name.isalpha():
        raise TypeError("Имя студента должно быть строкой")
    print(f"Оценка студента ____: _____")
except KeyError:
    print(f"Студент _____ не найден")
except TypeError as e:
    print(e)
```

5. Цикл с обработкой исключений

- Создайте список из 5 элементов, включая числа и строки.
- Используя цикл `for`, попробуйте разделить число 100 на каждый элемент списка.
- Обработайте исключения:
 - Если элемент равен 0, выведите сообщение: "Деление на ноль невозможно".
 - Если элемент не является числом, выведите сообщение: "Элемент [элемент] не является числом".
- Выведите результат деления для корректных элементов.

```
mixed_list = [10, 0, "abc", 5, 20]

for item in mixed_list:
    try:
```

```

result = _____
print(f"Результат деления 100 _____")
except _____:
    print("Деление на ноль невозможно")
except TypeError:
    print(f"Элемент _____ не является числом")

```

- Для чего необходимо обрабатывать исключения?

6. На основании исходных данных об успеваемости студентов проставьте им оценки и назначьте стипендию. Выполните задание используя цикл for, а также операторы case и match.

- Условия проставления оценок:
 - Отл – 90 и баллов
 - Хор – от 80 до 89 баллов
 - Уд – от 60 до 80 баллов
 - Неуд – менее 60 баллов
- Условия начисления стипендии:
 - Отл – 8000
 - Хор – 6000
 - Уд – 0
 - Неуд – 0
- Результат необходимо вывести в виде списка структуры:
 - stud_scholarship=[[имя, балл, стипендия, оценка], ...]

```

stud_grade=[["Vasia",99],["Petia",85],["Mary",90],["Pavel",
55],["Fedia",33]]
stud_scholarship=[]
for i in range(_____):
    match (stud_grade[____]):
        case (name,grade) if ____ stud_grade[____][____] ____:
            stud_scholarship.____([name,grade,8000,"отл"])
        case (name,grade) if ____ stud_grade[____][____] ____:
            stud_scholarship.____([name,grade,6000,"хор"])
        case (name,grade) if ____ stud_grade[____][____] ____:
            stud_scholarship.____([name,grade,0,"уд"])

```

```
        case (name, grade) if ____ stud_grade[____][____] ____:
            stud_scholarship.____([name, grade, 0, "неуд"])
print(stud_scholarship)
```

- Возможно ли решить эту задачу без применения оператора match?
- Предложите свой вариант решения данной задачи.

7. Напишите программу для нахождения среднего значения по медиане.

```
test_list = [8, 17, 4, 9, 10, 5]
print(test_list)

test_list.____()
mid = len(test_list) ____
res = (test_list[mid] + test_list[~mid]) ____

print("Среднее: " + str(____))
print("Среднее по медиане: " + str(____))
```

8. Напишите программу для сортировки массива методом пузырька.

Метод пузырька — это простой алгоритм сортировки, который работает путём последовательного сравнения соседних элементов массива и их обмена местами, если они находятся в неправильном порядке. Процесс продолжается до тех пор, пока массив не будет полностью отсортирован.

Основные шаги метода пузырька:

1. Сравниваем два соседних элемента массива. Если первый элемент больше второго, то меняем их местами.
2. Повторяем шаг 1 для всех последующих пар элементов, двигаясь слева направо по массиву.
3. После первого прохода по массиву самый большой элемент «всплывёт» в конец массива (отсюда название метода).
4. Повторяем шаги 1–3 для неотсортированной части массива до тех пор, пока все элементы не будут упорядочены.

9. Напишите программу для сортировки массива методом рекурсии.

```
# Функция реализующая метод рекурсии (быстрая сортировка)
def quicksort(arr):

    # Обработка случая, когда массив пуст или содержит один член
    if len(____) <= 1:
        return ____
    else:

        # Определение опорной точки "pivot" и разбиение массива на
        # правую и левую части
        pivot = arr[____]
        left = [x for x in arr[____] if x < pivot]
        right = [x for x in arr[____] if x >= pivot]

        # Рекурсивный вызов функции для правой и левой части
        return ____ (left) + [____] + ____ (right)

    # Использование функции быстрой сортировки
arr = [1, 7, 4, 1, 10, 9, -2]
sorted_arr = ____ (____)
print("Массив отсортирован по возрастанию:")
print(____)
```

- Почему сортировка методом рекурсии называется «быстрой»?
- В чем выражается рекурсия представленного выше кода?

10. Допишите в файл программы для обработки формул в формате SMILES дополнительный код, который будет сравнивать символы из списка с разделителями, выдавать все разделители из списка и их индекс.

```
if let in delimiters:
    if let == "[": print("найден разделитель [")
    elif let == "]": print("найден разделитель ]")

...

```

***Данный файл в дальнейшем будет дорабатываться. ОБЯЗАТЕЛЬНО СОХРАНИТЕ ЕГО! В результате доработок окончательный проект будет содержать обработку химической формулы в формате SMILES и первичный анализ соединения.**

Вопросы для самоконтроля

- В чем разница между `if-elif-else` и `match-case` (если изучали)?
- Какие типы циклов существуют в Python? В чем их отличия?
- Как работает `break` и `continue` в циклах?
- Какие исключения вы знаете? Как обрабатываются исключения в Python?
- Как можно обработать несколько типов исключений в одном блоке `try-except`?